MySQL Slow Query and Pool Waiting - Sanitized Incident Review

INC-MYSQL-2026-07-B | Owner: Payments Reliability
Internal training fixture - identifiers and traffic values sanitized
Reviewed: 2026-07-21 | Trace: INC-MYSQL-2026-07-B / CR-MYSQL-2026-044

1. Executive Summary and Impact

On 2026-07-06, payment-service checkout latency increased after release rc3 changed ORM eager loading for the order report query. The new query plan read far more rows, held MySQL connections longer, and caused application pool_waiting during peak traffic.

The customer-visible window lasted 19 minutes. Payment P95 rose from 610 ms to 5.2 s, timeout rate peaked at 9.6%, and 3.1% of payment attempts were retried. Idempotency controls prevented duplicate charges.

This document records a historical diagnosis. Deploy correlation alone is not root-cause proof; the release evidence was accepted only after matching the SQL digest, connection occupancy, and incident-window latency.

Scope	Observed impact
Service	payment-service / checkout
Window	2026-07-06 10:05-10:24 UTC
Peak database signals	slow_queries=18, active_connections=188/200
Application signal	pool_waiting=6, checkout timeout
Integrity	No confirmed duplicate charge or lost payment event

2. Incident Timeline

Evidence was aligned to the incident window and separated into deployment, database, application, and customer-impact signals.

UTC	Event and evidence
09:42	payment-api rc3 changed checkout query loading
10:05	PaymentLatencyHigh alert began firing
10:08	slow_queries=18 and active_connections=188/200
10:10	Loki recorded digest payment_report_join_v3 and pool waits
10:13	Read-only EXPLAIN showed high row estimate and temporary table
10:15	Incident commander approved disabling report feature on canary
10:18	Canary pool_waiting fell to zero; timeout rate declined
10:24	Core payment SLO recovered
10:32	rc4 completed with report feature disabled

3. Evidence Register

Evidence was captured before any database mutation. Read-only EXPLAIN and application metrics were used to separate a query-plan regression from host, network, and permanent connection-leak hypotheses.

Evidence	Observation	Supports	Limit
Prometheus	Connections 188/200; pool_waiting=6	Pool pressure	Does not name SQL
Slow query digest	payment_report_join_v3 appeared after rc3	Query correlation	Digest omits literals
Read-only EXPLAIN	Wide join and temporary table	Plan regression	Estimate, not runtime trace
Deploy history	ORM eager loading changed at 09:42	Candidate trigger	Correlation alone
Canary	Disabling feature cleared pool waits	Causal reversal	10% traffic scope

4. Hypothesis Review and Root Cause

Hypothesis A - database host CPU exhaustion: rejected. CPU increased only after the slow digest volume rose and remained below the saturation threshold.

Hypothesis B - network latency between application and MySQL: rejected. TCP connect latency and packet loss stayed near baseline.

Hypothesis C - connection leak: not primary. Connections returned to the pool after requests completed; their hold time, not permanent leakage, increased.

Hypothesis D - rc3 query-plan regression: confirmed. The digest appeared after deployment, EXPLAIN showed a wider join and temporary table, connection hold time increased, and disabling the feature reversed pool_waiting.

Root cause: an ORM eager-loading change expanded the checkout report join. The query held connections long enough to consume the application pool, creating queueing and request timeouts.

Diagnostic conclusion: the confirmed hypothesis is supported by the new digest, read-only plan evidence, connection hold time, and causal reversal. Rejected hypotheses are retained as negative evidence for future incident comparison.

5. Response, Approval, and Follow-up

The initial response captured the SQL digest, transaction state, pool metrics, and deploy history before proposing a change. EXPLAIN was run through a read-only path. No index, pool-size, SQL, or restart change was executed by the agent.

Approved change CR-MYSQL-2026-044 disabled the report feature on a 10% canary. The payments service owner approved the application change; the DBA approved the later covering-index plan. Rollback criteria included increased lock wait, replication lag above 10 s, or payment error rate above 2%.

Recovery required pool_waiting=0, active connections below 70% of capacity, P95 below 1 s, no duplicate-charge signal, and 30 minutes of stable observation.

Recovery verification was owned by Payments Reliability and the DBA. The incident record was reviewed on 2026-07-21 and linked to the approval, query regression test, and covering-index action tickets.

Record	Change or action	Owner / approver	Result / due
Approval	CR-MYSQL-2026-044, disable report on 10% canary	Payments Owner + DBA	Approved 10:15 UTC
Action	Add query-plan regression test	Payments	2026-07-22 / CI fixture
Action	Create covering index after review	DBA	2026-07-25 / EXPLAIN + canary
Action	Alert on pool wait and hold time	SRE	2026-07-24 / alert drill
Action	Add release-to-digest correlation	Observability	2026-07-31 / dashboard